HavenBridge TLS, HTTPS, cert-manager, and Private PKI

Purpose

This document explains how TLS and HTTPS are implemented in the HavenBridge Kubernetes platform.

It is intentionally written as both:

- implementation documentation;
- a learning guide;
- a troubleshooting reference; and
- an interview-preparation document.

The HavenBridge TLS architecture uses:

```
cert-manager
    ↓
SelfSigned ClusterIssuer
    ↓
HavenBridge Root CA
    ↓
CA ClusterIssuer
    ↓
havenbridge.lab server certificate
    ↓
Kubernetes TLS Secret
    ↓
Traefik HTTPS Gateway listener
    ↓
https://havenbridge.lab
```

The final validated application endpoint is:

```
https://havenbridge.lab
```

The readiness endpoint is:

```
https://havenbridge.lab/health/ready
```

The final validation returned:

```
HTTP/2 200
{"status":"ready"}
```

Why HavenBridge Needs HTTPS

HTTP sends application traffic without TLS encryption.

Conceptually:

```
Client
   ↓
HTTP
   ↓
plaintext traffic
   ↓
Server
```

HTTPS means:

```
HTTP
+
TLS
=
HTTPS
```

With HTTPS:

```
Client
   ↓
encrypted TLS connection
   ↓
Traefik
   ↓
application routing
```

TLS gives HavenBridge three important security properties.

Encryption

Other systems on the network should not be able to simply read application traffic in plaintext.

Authentication

The server presents a certificate that identifies:

```
havenbridge.lab
```

The client can verify that it is communicating with the expected server.

Integrity

TLS helps prevent data from being silently altered while moving between the client and the server.

HTTP Versus HTTPS

The simple distinction is:

```
HTTP
= application protocol without TLS protection

HTTPS
= HTTP carried inside a TLS-protected connection
```

For HavenBridge:

```
HTTP
TCP/80
```

is used only to redirect users.

The application itself is served using:

```
HTTPS
TCP/443
```

The final behavior is:

```
http://havenbridge.lab
        ↓
301 Moved Permanently
```

```
        ↓
https://havenbridge.lab
```

Encryption Is Not the Same as Trust

One of the most important lessons from this implementation is:

```
Encryption ≠ Trust
```

During the first HTTPS test, this command:

```
curl -v https://havenbridge.lab/health/ready
```

failed with:

```
SSL certificate problem:
unable to get local issuer certificate
```

However:

```
curl -vk https://havenbridge.lab/health/ready
```

successfully returned:

```
HTTP/2 200
{"status":"ready"}
```

This proved:

```
HTTPS listener        ✅
TLS handshake         ✅
Certificate presented ✅
Application routing   ✅
API response           ✅

Client trust          ❌
```

The problem was not encryption.

The problem was that Syrus did not yet trust the private HavenBridge Root CA.

After installing the Root CA into the client trust store:

```
curl https://havenbridge.lab/health/ready
```

worked normally.

What Is PKI?

PKI means:

```
Public Key Infrastructure
```

It is the system of:

- certificate authorities;
- certificates;
- public keys;
- private keys;
- trust relationships; and
- certificate lifecycle management

used to establish trusted encrypted communication.

The HavenBridge PKI hierarchy is:

```
HavenBridge Root CA
       ↓
HavenBridge CA issuer
       ↓
havenbridge.lab certificate
```

What Is a Certificate Authority?

A Certificate Authority, or CA, signs certificates.

The CA effectively says:

> I have signed this certificate and confirm that it belongs to the identity specified in the certificate.

For HavenBridge:

```
HavenBridge Root CA
        ↓ signs
havenbridge.lab certificate
```

If a client trusts the HavenBridge Root CA, it can trust certificates signed by that CA, subject to normal certificate validation.

What Is a Root CA?

The Root CA is the top of the HavenBridge certificate trust hierarchy.

Its identity is:

```
Organization:
HavenBridge

Common Name:
HavenBridge Root CA
```

The Root CA is self-signed.

Its certificate inspection showed:

```
subject=O = HavenBridge, CN = HavenBridge Root CA

issuer=O = HavenBridge, CN = HavenBridge Root CA
```

The subject and issuer are identical because the Root CA signs its own certificate.

Conceptually:

```
HavenBridge Root CA
Subject = HavenBridge Root CA
Issuer  = HavenBridge Root CA
```

The validated Root CA lifetime is approximately ten years:

```
notBefore:
Aug 9 2026

notAfter:
Aug 6 2036
```

Why Use a Private Root CA?

The hostname:

```
havenbridge.lab
```

is an internal homelab hostname.

It is not intended to be a public Internet DNS name.

Instead of using a public CA, HavenBridge uses its own private CA.

This gives the project experience with:

- PKI;
- certificate issuance;
- private trust distribution;
- TLS Secrets;
- certificate lifecycle management;
- TLS termination; and
- client trust configuration.

The tradeoff is important:

```
Public CA
→ commonly trusted automatically

Private HavenBridge CA
→ clients must explicitly trust it
```

What Is cert-manager?

cert-manager is a Kubernetes certificate-management controller.

It extends Kubernetes with certificate-related resources and automates certificate issuance and maintenance.

Before cert-manager was installed, Kubernetes understood standard objects such as:

```
Pod
Deployment
Service
Secret
ConfigMap
```

After installing cert-manager, Kubernetes also understands objects such as:

```
Certificate
CertificateRequest
Issuer
ClusterIssuer
```

cert-manager therefore turns certificate management into Kubernetes declarative configuration.

Instead of manually generating every key and certificate with OpenSSL, HavenBridge defines the desired certificate state in YAML.

---

cert-manager Installation

cert-manager was installed into the Kubernetes cluster using Helm.

The validated environment was:

```
Kubernetes:
v1.36

Helm:
v4.2.3

cert-manager:
v1.21.0
```

The chart was validated before installation using:

```
helm show chart
  oci://quay.io/jetstack/charts/cert-manager
  --version v1.21.0
```

The installation used:

```
helm install
  cert-manager
  oci://quay.io/jetstack/charts/cert-manager
  --version v1.21.0
  --namespace cert-manager
  --create-namespace
  --set crds.enabled=true
```

---

cert-manager Namespace

cert-manager runs in:

```
namespace:
cert-manager
```

The installation created the core components:

```
cert-manager
cert-manager-cainjector
cert-manager-webhook
```

All three were validated as:

```
1/1 Running
```

---

cert-manager Components

cert-manager controller

The main controller watches certificate-related Kubernetes resources.

For example:

```
Certificate
Issuer
ClusterIssuer
CertificateRequest
```

and reconciles them into the desired certificate state.

cert-manager webhook

The webhook validates and processes cert-manager API objects when they are submitted to the Kubernetes API.

cert-manager cainjector

The CA injector helps place CA certificate data where Kubernetes resources require it.

What Is a CRD?

CRD means:

```
Custom Resource Definition
```

A CRD extends the Kubernetes API with new object types.

The required cert-manager CRDs were validated:

```
certificates.cert-manager.io

issuers.cert-manager.io

clusterissuers.cert-manager.io
```

This means Kubernetes can now understand YAML such as:

```
kind: Certificate
```

and:

```
kind: ClusterIssuer
```

The Four HavenBridge TLS YAML Files

The private PKI is created using four primary YAML manifests.

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
```

contains:

```
selfsigned-clusterissuer.yaml

havenbridge-root-ca-certificate.yaml

havenbridge-ca-clusterissuer.yaml

havenbridge-certificate.yaml
```

Their dependency chain is:

```
1. selfsigned-clusterissuer.yaml
             ↓

2. havenbridge-root-ca-certificate.yaml
             ↓

3. havenbridge-ca-clusterissuer.yaml
             ↓

4. havenbridge-certificate.yaml
```

An easy way to remember the sequence is:

```
Bootstrap
   ↓
Root authority
   ↓
Reusable signer
   ↓
Website certificate
```

1. SelfSigned ClusterIssuer

File:

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
selfsigned-clusterissuer.yaml
```

The object is:

```
ClusterIssuer:
havenbridge-selfsigned
```

Its purpose is:

```
BOOTSTRAP ONLY
```

It exists to create the first Root CA certificate.

A SelfSigned ClusterIssuer is not itself the Root CA.

Think of it as:

```
SelfSigned ClusterIssuer
= machine capable of creating the first master stamp
```

Nothing is automatically issued merely because the issuer exists.

A `Certificate` resource must request a certificate from it.

Why Use a ClusterIssuer?

cert-manager supports both:

```
Issuer
```

and:

```
ClusterIssuer
```

An `Issuer` is namespace-scoped.

A `ClusterIssuer` is cluster-scoped.

Conceptually:

```
Issuer
    ↓
usable by Certificate resources
within one namespace
```

versus:

```
ClusterIssuer
    ↓
usable across namespaces
```

HavenBridge uses ClusterIssuers because certificate consumers exist across platform namespaces such as:

```
cert-manager
traefik
havenbridge
```

---

2. HavenBridge Root CA Certificate

File:

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
havenbridge-root-ca-certificate.yaml
```

The resource is:

```
Certificate:
havenbridge-root-ca
```

in:

```
namespace:
cert-manager
```

This is the object that actually requests creation of the Root CA certificate.

Its key relationship is:

```
issuerRef:
havenbridge-selfsigned
```

Therefore:

```
havenbridge-selfsigned
        ↓ signs
havenbridge-root-ca
```

---

isCA: true

The Root CA Certificate contains:

```
isCA: true
```

This is critical.

It means:

> This certificate is intended to act as a Certificate Authority.

A normal server certificate should not have CA authority.

Conceptually:

```
Certificate
+
isCA: true
=
CA certificate
```

Root CA Key Configuration

The HavenBridge Root CA uses:

```
RSA
4096-bit key
```

The Root CA private key is intentionally stronger and longer lived than the application leaf certificate.

The configuration also uses:

```
rotationPolicy: Never
```

This does not mean the Root CA should literally never be rotated.

It means Root CA key rotation should be a deliberate PKI maintenance operation rather than occurring automatically as part of normal certificate reissuance.

Changing a Root CA can affect every client that trusts it.

Root CA Lifetime

The Root CA uses:

```
duration:
87600h
```

which is approximately:

```
10 years
```

Renewal begins approximately:

```
1 year
```

before expiration.

The long lifetime reflects the Root CA's role as a trust anchor.

Leaf/server certificates should have much shorter lifetimes.

Root CA Secret

After cert-manager issued the Root CA, it created:

```
Secret:
havenbridge-root-ca
```

in:

```
namespace:
cert-manager
```

The Secret type is:

```
kubernetes.io/tls
```

The Root CA Certificate was validated as:

```
READY=True

SECRET=havenbridge-root-ca

ISSUER=havenbridge-selfsigned
```

This Secret contains the Root CA certificate and signing key material used by the next issuer.

---

3. HavenBridge CA ClusterIssuer

File:

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
havenbridge-ca-clusterissuer.yaml
```

The resource is:

```
ClusterIssuer:
havenbridge-ca
```

This is the normal HavenBridge certificate signer.

Its configuration references:

```
Secret:
havenbridge-root-ca
```

Conceptually:

```
Secret: havenbridge-root-ca
       ↓
contains CA certificate + private key
       ↓
ClusterIssuer: havenbridge-ca
       ↓
can sign HavenBridge certificates
```

The `havenbridge-ca` ClusterIssuer was validated:

```
READY=True
```

Why Not Keep Using the SelfSigned Issuer?

The SelfSigned issuer was only required to bootstrap the Root CA.

After the Root CA exists:

```
havenbridge-selfsigned
       ↓
job complete
```

Normal certificates use:

```
havenbridge-ca
```

instead.

The chain therefore becomes:

```
SelfSigned issuer
      ↓ bootstrap
Root CA
      ↓
CA issuer
      ↓
normal certificates
```

---

4. havenbridge.lab Server Certificate

File:

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
havenbridge-certificate.yaml
```

The resource is:

```
Certificate:
havenbridge-tls
```

It exists in:

```
namespace:
traefik
```

The resulting Secret is also named:

```
havenbridge-tls
```

Why Is havenbridge-tls in the Traefik Namespace?

The final certificate is consumed by Traefik.

The Gateway exists in:

```
namespace:
traefik
```

The TLS Secret used by the Gateway is:

```
Secret:
havenbridge-tls
```

also in:

```
namespace:
traefik
```

The design is:

```
cert-manager
      ↓ issues
Certificate: havenbridge-tls
namespace: traefik
      ↓
Secret: havenbridge-tls
namespace: traefik
      ↓
Traefik Gateway HTTPS listener
```

cert-manager does not need to run in the same namespace as every Certificate it manages.

The cert-manager controllers run centrally while Certificate objects can exist in workload namespaces.

Server Certificate DNS Identity

The leaf certificate includes:

```
DNS:
havenbridge.lab
```

The client uses this identity to verify that the certificate matches the hostname requested.

For example:

```
Client connects to:
havenbridge.lab

Certificate SAN contains:
havenbridge.lab
```

Therefore hostname validation can succeed.

Root CA Versus Server Certificate

This distinction is extremely important.

Root CA

```
havenbridge-root-ca
```

Purpose:

```
sign certificates
```

Server certificate

```
havenbridge-tls
```

Purpose:

```
identify havenbridge.lab to clients
```

Conceptually:

```
Root CA
   ↓ signs
Server certificate
   ↓ presented by
Traefik
```

---

ca.crt Versus tls.crt Versus tls.key

A Kubernetes TLS-related Secret may contain data such as:

```
ca.crt
tls.crt
tls.key
```

An easy way to remember them is:

ca.crt

```
CA public certificate
```

Purpose:

```
establish who issued/signed certificates
```

Memory hook:

```
passport office's official stamp
```

tls.crt

```
server public certificate
```

Purpose:

```
identifies the server
```

For HavenBridge:

```
havenbridge.lab
```

Memory hook:

```
HavenBridge's passport
```

tls.key

```
server private key
```

Purpose:

```
proves possession of the certificate's corresponding private key
```

Memory hook:

```
HavenBridge's secret signature
```

Which Certificate Files Are Safe to Share?

Generally:

```
ca.crt
→ public certificate
→ safe to distribute for trust purposes

tls.crt
→ public server certificate
→ presented to clients

tls.key
→ PRIVATE
→ must be protected
```

The private key should never be committed to Git or casually copied between systems.

For the Root CA, protecting the signing private key is especially important because compromise of that key undermines the trust hierarchy.

Certificate Chain

The HavenBridge trust chain can be visualized as:

```
HavenBridge Root CA
Subject:
HavenBridge Root CA

Issuer:
HavenBridge Root CA

CA:
TRUE
        |
        | signs
        v
havenbridge.lab certificate

Issuer:
HavenBridge Root CA

DNS:
havenbridge.lab

CA:
FALSE
```

The client trusts:

```
HavenBridge Root CA
```

and therefore accepts the certificate signed by that CA for:

```
havenbridge.lab
```

Certificate Lifecycle

The Root CA is long-lived.

The server certificate is intentionally shorter-lived.

The server certificate configuration uses approximately:

```
duration:
2160h
```

which is:

```
90 days
```

and:

```
renewBefore:
720h
```

which is:

```
30 days
```

Conceptually:

```
Certificate issued
       ↓
90-day validity
       ↓
30 days remain
       ↓
cert-manager begins renewal
```

This demonstrates one of the main advantages of cert-manager:

```
certificate lifecycle automation
```

Server Private-Key Rotation

The server certificate uses:

```
rotationPolicy:
Always
```

The Root CA uses:

```
rotationPolicy:
Never
```

The distinction is intentional.

Rotating a leaf/server key is normal certificate hygiene.

Rotating the Root CA key changes the trust anchor and requires more careful planning.

TLS Secret Validation

The final Traefik TLS Secret was validated using:

```
kubectl get secret havenbridge-tls
  --namespace traefik
  --output wide
```

The result showed:

```
NAME:
havenbridge-tls

TYPE:
kubernetes.io/tls

DATA:
3
```

This proved the final certificate material was available for the Gateway HTTPS listener.

How TLS Connects to Traefik

The final TLS Secret is referenced by the Traefik Gateway.

Conceptually:

```
Certificate:
havenbridge-tls
        ↓
Secret:
havenbridge-tls
        ↓
Gateway listener:
websecure
        ↓
TLS termination
        ↓
HTTPS traffic
```

The Gateway configuration includes:

```
protocol:
HTTPS

mode:
Terminate

certificateRefs:
havenbridge-tls
```

What Is TLS Termination?

TLS termination is the point where encrypted traffic is decrypted.

In HavenBridge:

```
Client
   ||
   || encrypted HTTPS
   ||
   v
Traefik
   ↓
TLS terminates here
   ↓
internal routing
```

Traefik handles:

- TLS handshake;
- certificate presentation;
- private-key operations; and
- decryption of incoming HTTPS requests.

The application Pod itself does not directly perform the external TLS handshake.

HavenBridge HTTPS Port Flow

The external HTTPS path is:

```
Client
   ↓
TCP/443
   ↓
Traefik LoadBalancer Service
   ↓
servicePort 443
   ↓
targetPort websecure
   ↓
Traefik containerPort 8443
   ↓
Gateway HTTPS listener
```

Easy version:

```
443 → websecure:8443
```

Complete HTTPS Request Flow

```
Client
        ↓
https://havenbridge.lab
        ↓
DNS
        ↓
172.16.10.40
        ↓
```

```
MetalLB
        ↓
Traefik LoadBalancer Service
        ↓
TCP/443
        ↓
websecure:8443
        ↓
Gateway HTTPS listener
        ↓
TLS handshake
        ↓
Traefik presents havenbridge.lab certificate
        ↓
Client validates certificate chain
        ↓
TLS termination
        ↓
HTTPRoute
        ↓
havenbridge-api Service :80
        ↓
EndpointSlice
        ↓
Ready API Pod :8000
        ↓
FastAPI
        ↓
HTTP/2 200
```

HTTP to HTTPS Redirect

HTTP traffic is no longer used to serve the application directly.

The final flow is:

```
HTTP :80
   ↓
web:8000
   ↓
havenbridge-http-redirect
   ↓
301 Moved Permanently
   ↓
```

```
HTTPS :443
   ↓
websecure:8443
   ↓
TLS
   ↓
application
```

Validation showed:

```
HTTP/1.1 301 Moved Permanently

Location:
https://havenbridge.lab/health/ready
```

Following the redirect produced:

```
Final URL:
https://havenbridge.lab/health/ready

HTTP Code:
200

Redirects:
1
```

Why 301?

The redirect uses:

```
301 Moved Permanently
```

because HavenBridge intends HTTPS to be the permanent application access method.

Simple distinction:

```
300
= multiple choices

301
```

```
= permanent redirect

302
= temporary redirect
```

For API write operations such as POST or PUT, clients should call HTTPS directly rather than relying on redirects.

Initial Client Trust Failure

Before the Root CA was installed on Syrus, the first HTTPS test produced:

```
SSL certificate problem:
unable to get local issuer certificate
```

The connection reached:

```
havenbridge.lab
172.16.10.40
TCP/443
```

and Traefik presented a certificate.

This was therefore a client trust problem, not a routing problem.

Diagnostic curl -k Test

The temporary diagnostic command was:

```
curl -vk https://havenbridge.lab/health/ready
```

The `-k` option tells curl:

> Continue with HTTPS but skip normal certificate trust verification.

This should only be used for troubleshooting.

It should not become the normal method of accessing the application.

The test successfully returned:

```
HTTP/2 200
{"status":"ready"}
```

This proved the TLS listener and application routing were functioning before client trust was configured.

Installing the Root CA on Syrus

The public Root CA certificate was exported from Kubernetes.

It was stored temporarily as:

```
/home/alabi/havenbridge-root-ca.crt
```

The certificate was inspected using:

```
openssl x509
  -in /home/alabi/havenbridge-root-ca.crt
  -noout
  -subject
  -issuer
  -dates
```

The output confirmed:

```
subject=
O = HavenBridge,
CN = HavenBridge Root CA

issuer=
O = HavenBridge,
CN = HavenBridge Root CA
```

Ubuntu System Trust Store

The Root CA certificate was added to Syrus's Ubuntu trust store under:

```
/usr/local/share/ca-certificates/
```

and the CA database was updated using:

```
sudo update-ca-certificates
```

After that:

```
curl https://havenbridge.lab/health/ready
```

worked without:

```
-k
```

This demonstrated proper certificate validation.

Browser Trust

An important lesson was that operating-system trust and browser trust are not always identical.

Command-line curl trusted the certificate after the Ubuntu CA store was updated.

However, the Chromium-based browser still displayed:

```
Not secure
```

even though the HTTPS page loaded.

This meant:

```
TLS connection          ✅
Certificate presented   ✅
Application response    ✅
Browser trust           ❌
```

Chromium and Brave NSS Trust

The browser was using an NSS database located at:

```
/home/alabi/.pki/nssdb
```

The HavenBridge Root CA therefore also needed to be imported into that browser trust database.

This demonstrates an important real-world lesson:

```
A private CA must be trusted by every client trust store
that needs to validate certificates issued by that CA.
```

That may include:

- operating system trust;
- browsers;
- Java trust stores;
- application-specific CA bundles;
- containers; or
- other client systems.

Building Analogy for PKI

The certificate chain can be remembered using an office/government analogy.

```
SelfSigned ClusterIssuer
= machine capable of creating the first master stamp

Root CA Certificate
= the master stamp

havenbridge-ca ClusterIssuer
= authorized office allowed to use the master stamp

havenbridge.lab certificate
= official ID issued to HavenBridge

ca.crt
= public copy of the authority's official stamp

tls.crt
= HavenBridge's official ID

tls.key
= HavenBridge's secret signature
```

```
Traefik
= secure receptionist presenting the ID to visitors
```

Client trust works like:

```
Visitor trusts official authority
        ↓
authority signed HavenBridge ID
        ↓
visitor trusts HavenBridge ID
```

Simple Definitions to Remember

TLS

```
Encryption and identity protection for network communication.
```

Memory hook:

```
"Secure envelope for HTTP."
```

HTTPS

```
HTTP protected by TLS.
```

Memory hook:

```
"Secure version of HTTP."
```

Certificate

```
Public digital identity that connects a name to a public key.
```

Memory hook:

```
"Digital passport."
```

Private key

```
Secret cryptographic key proving ownership of a certificate.
```

Memory hook:

```
"Secret signature."
```

Certificate Authority

```
Trusted authority that signs certificates.
```

Memory hook:

```
"Passport office."
```

Root CA

```
Top-level trust authority in the certificate chain.
```

Memory hook:

```
"Master authority."
```

cert-manager

```
Kubernetes controller that automates certificate management.
```

Memory hook:

```
"Kubernetes certificate department."
```

ClusterIssuer

```
Cluster-wide certificate signer managed by cert-manager.
```

Memory hook:

```
"Certificate authority available across namespaces."
```

TLS termination

```
The point where HTTPS traffic is decrypted.
```

Memory hook:

```
"Secure envelope opened at Traefik."
```

Security Considerations

Protect private keys

Never commit:

```
tls.key
```

or Root CA signing keys to Git.

Protect Root CA signing material

Compromise of the Root CA private key could allow an attacker to create certificates trusted by HavenBridge clients.

Use TLS for external traffic

The public application path should use:

```
https://havenbridge.lab
```

Redirect HTTP

HTTP remains available only to redirect users to HTTPS.

Private CA distribution

Only trusted systems should receive the HavenBridge Root CA certificate.

The Root CA certificate itself is public trust material, but its distribution determines which clients trust HavenBridge-issued certificates.

Availability Considerations

cert-manager is responsible for certificate issuance and renewal.

An already-issued certificate remains stored as a Kubernetes Secret.

Therefore temporary cert-manager controller failure does not necessarily make an already-running HTTPS listener immediately stop serving its current certificate.

However, extended cert-manager failure could affect:

- renewal;
- new certificate issuance; and
- certificate lifecycle operations.

The cert-manager Pods were initially observed on:

```
eph-worker02
```

Controller HA may be revisited in a later platform-hardening phase.

Failure Scenario: cert-manager Down

Possible effect:

```
Existing certificate
may continue working

New certificate issuance
may fail

Certificate renewal
may be delayed
```

Troubleshooting:

```
kubectl get pods
  --namespace cert-manager
```

Failure Scenario: Missing Root CA Secret

If:

```
havenbridge-root-ca
```

is missing, the CA ClusterIssuer may no longer have access to the signing material required for certificate issuance.

Check:

```
kubectl get secret havenbridge-root-ca
  --namespace cert-manager
```

Failure Scenario: CA ClusterIssuer Not Ready

Check:

```
kubectl get clusterissuer havenbridge-ca
```

Expected:

```
READY=True
```

If false:

```
kubectl describe clusterissuer havenbridge-ca
```

Failure Scenario: Server Certificate Not Ready

Check:

```
kubectl get certificate havenbridge-tls
  --namespace traefik
  --output wide
```

Expected:

```
READY=True
```

If false:

```
kubectl describe certificate havenbridge-tls
  --namespace traefik
```

Failure Scenario: TLS Secret Missing

Check:

```
kubectl get secret havenbridge-tls
  --namespace traefik
```

If the Gateway references a Secret that does not exist, the HTTPS listener may fail to resolve its certificate reference correctly.

Failure Scenario: Gateway Certificate Reference Error

Inspect:

```
kubectl describe gateway havenbridge-gateway
  --namespace traefik
```

Look at:

```
Accepted
ResolvedRefs
Programmed
```

A certificate reference problem may appear as:

```
ResolvedRefs=False
```

Failure Scenario: curl Reports Unknown CA

Example:

```
curl: (60)
SSL certificate problem:
unable to get local issuer certificate
```

Interpretation:

```
TLS may be working,
but the client does not trust the issuing CA.
```

Check whether the HavenBridge Root CA exists in the client's trust store.

Failure Scenario: Browser Says Not Secure

If:

```
curl HTTPS works
```

but the browser still shows:

```
Not secure
```

investigate the browser's certificate trust database.

For Chromium/Brave on Syrus, the NSS database was:

```
/home/alabi/.pki/nssdb
```

Troubleshooting Order

A structured TLS troubleshooting order is:

```
DNS
 ↓
TCP/443
 ↓
Traefik Service
 ↓
websecure entrypoint
 ↓
Gateway listener
 ↓
TLS Secret
 ↓
Certificate
 ↓
Issuer
 ↓
Root CA
 ↓
Client trust
 ↓
HTTPRoute
 ↓
Service
 ↓
API
```

This prevents random troubleshooting.

Validation Commands

Check cert-manager Pods

```
kubectl get pods
  --namespace cert-manager
  --output wide
```

Check certificate CRDs

```
kubectl get crd
  certificates.cert-manager.io
  issuers.cert-manager.io
  clusterissuers.cert-manager.io
```

Check ClusterIssuers

```
kubectl get clusterissuer
```

Expected:

```
havenbridge-selfsigned   True
havenbridge-ca           True
```

Check all certificates

```
kubectl get certificate -A -o wide
```

Expected certificate resources include:

```
cert-manager/havenbridge-root-ca
traefik/havenbridge-tls
```

Both should report:

```
READY=True
```

Check Root CA Secret

```
kubectl get secret havenbridge-root-ca
  --namespace cert-manager
```

Check server TLS Secret

```
kubectl get secret havenbridge-tls
  --namespace traefik
```

Inspect Root CA

```
openssl x509
  -in /home/alabi/havenbridge-root-ca.crt
  -noout
  -subject
  -issuer
  -dates
```

Check Gateway TLS listener

```
kubectl describe gateway havenbridge-gateway
  --namespace traefik
```

Test HTTPS

```
curl -i https://havenbridge.lab/health/ready
```

Expected:

```
HTTP/2 200
{"status":"ready"}
```

Test HTTP redirect

```
curl -s -o /dev/null -D -
  http://havenbridge.lab/health/ready
  | grep -Ei '^(HTTP/|Location:)'
```

Expected:

```
HTTP/1.1 301 Moved Permanently

Location:
https://havenbridge.lab/health/ready
```

Follow redirect

```
curl -sS -L
  -o /dev/null
```

```
  -w 'Final URL: %{url_effective}\nHTTP Code: %{http_code}\nRedirects: %
{num_redirects}\n'
  http://havenbridge.lab/health/ready
```

Expected:

```
Final URL:
https://havenbridge.lab/health/ready

HTTP Code:
200

Redirects:
1
```

Operational Certificate Checks

Certificate readiness

```
kubectl get certificate -A
```

Certificate details

```
kubectl describe certificate havenbridge-tls
  --namespace traefik
```

Issuer readiness

```
kubectl get clusterissuer
```

cert-manager logs

```
kubectl logs
  --namespace cert-manager
  deployment/cert-manager
```

Webhook logs

```
kubectl logs
  --namespace cert-manager
  deployment/cert-manager-webhook
```

CA injector logs

```
kubectl logs
  --namespace cert-manager
  deployment/cert-manager-cainjector
```

Current Limitations

Private CA

The HavenBridge CA is not publicly trusted.

Every client that accesses HavenBridge must trust the Root CA separately.

This is appropriate for the lab but would require a broader trust-management strategy in an enterprise.

Root CA key storage

The Root CA private key exists in a Kubernetes Secret.

A more advanced production PKI might protect high-value CA signing keys using:

- dedicated PKI systems;
- cloud KMS;
- HSMs; or
- services such as Vault.

TLS terminates at Traefik

The current design protects:

```
Client → Traefik
```

with TLS.

The internal backend connection is not currently configured as application-level TLS.

CA disaster recovery

Loss of the Root CA private key would affect future certificate issuance and renewal.

A secure backup and recovery strategy for CA signing material should be considered in a more production-oriented implementation.

Interview Talking Points

What did you implement for TLS?

> I installed cert-manager in the Kubernetes cluster and created a private PKI for the HavenBridge lab. A SelfSigned ClusterIssuer bootstraps a private Root CA, the Root CA backs a CA ClusterIssuer, and that issuer signs the `havenbridge.lab` server certificate. cert-manager stores the final certificate and private key in a Kubernetes TLS Secret consumed by the Traefik Gateway HTTPS listener.

Why did you need a SelfSigned ClusterIssuer?

> I needed a way to bootstrap the first Root CA certificate. The SelfSigned ClusterIssuer is used only for that bootstrap operation. Once the Root CA exists, normal server certificates are signed by a CA-backed ClusterIssuer instead.

What is the difference between the Root CA and the server certificate?

> The Root CA is the trust anchor that signs certificates. The server certificate identifies `havenbridge.lab` and is presented by Traefik to clients during the TLS handshake.

Why is the Root CA Certificate in the cert-manager namespace?

> The Root CA Secret backs a cluster-scoped CA ClusterIssuer. cert-manager's cluster resource namespace is where the ClusterIssuer accesses that signing Secret.

Why is havenbridge-tls in the Traefik namespace?

> Traefik is the consumer of the server certificate. The Gateway HTTPS listener references the `havenbridge-tls` Secret, so the Certificate and resulting Secret live in the Traefik namespace.

What is TLS termination?

> TLS termination is where the encrypted client connection is decrypted. In HavenBridge, TLS terminates at Traefik. Traefik presents the `havenbridge.lab` certificate, completes the handshake, and then routes the request internally to the backend Service.

What is the difference between encryption and trust?

Encryption means the traffic is protected cryptographically. Trust means the client has verified that the certificate was issued by a CA it trusts and matches the hostname it requested. HavenBridge initially had working encryption but curl rejected the certificate until the private Root CA was added to the client's trust store.

Why did curl -k work before normal curl?

`curl -k` disables normal certificate verification. It allowed me to prove the TLS listener and application routing worked before configuring client trust. It was only a diagnostic step and was not kept as the normal access method.

Explain ca.crt, tls.crt, and tls.key.

`ca.crt` is the public CA certificate used to establish trust. `tls.crt` is the server's public certificate. `tls.key` is the corresponding private key and must be protected.

Why redirect HTTP to HTTPS?

The application should not normally serve unencrypted HTTP traffic. The HTTP listener returns a permanent redirect to HTTPS, while the HTTPS listener is responsible for actual application routing.

Certificate Chain Summary

The entire HavenBridge certificate chain can be remembered as:

```
SelfSigned ClusterIssuer
        ↓
bootstrap

Root CA Certificate
        ↓
trust anchor

Root CA Secret
        ↓
signing material

CA ClusterIssuer
        ↓
normal certificate signer

havenbridge.lab Certificate
```

```
        ↓
website identity

havenbridge-tls Secret
        ↓
certificate + server private key

Traefik HTTPS listener
        ↓
TLS termination

https://havenbridge.lab
```

Final TLS Architecture

```
                    HavenBridge Private PKI

                SelfSigned ClusterIssuer
                          ↓
                  HavenBridge Root CA
                          ↓
                    CA ClusterIssuer
                          ↓
               havenbridge.lab Certificate
                          ↓
                Secret: havenbridge-tls
                          |
                          |
                          v

Client
   |
   | HTTPS :443
   | TLS encrypted
   v
172.16.10.40
   |
   v
MetalLB
   |
   v
Traefik LoadBalancer Service
   |
   | 443 → websecure
```

```
   v
Traefik :8443
   |
   v
Gateway HTTPS Listener
   |
   | certificateRefs:
   | havenbridge-tls
   |
   | TLS terminates here
   v
HTTPRoute
   |
   v
havenbridge-api Service :80
   |
   v
EndpointSlice
   |
   v
Ready API Pod :8000
   |
   v
FastAPI
```

Related Files

TLS directory

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
```

SelfSigned ClusterIssuer

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
selfsigned-clusterissuer.yaml
```

Root CA Certificate

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
havenbridge-root-ca-certificate.yaml
```

CA ClusterIssuer

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
havenbridge-ca-clusterissuer.yaml
```

HavenBridge server certificate

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
havenbridge-certificate.yaml
```

TLS README

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/tls/
README.md
```

Traefik configuration

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/
traefik/values.yaml
```

Traefik README

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/
traefik/README.md
```

Backend HTTPS HTTPRoute

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/applications/
havenbridge/backend/httproute.yaml
```

HTTP redirect HTTPRoute

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/applications/
havenbridge/backend/httproute-http-redirect.yaml
```

TLS validation evidence

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/
evidence/tls-validation/
```

Platform README

```
/home/alabi/projects/havenbridge-ha-service-platform/kubernetes/platform/
README.md
```

Root project README

```
/home/alabi/projects/havenbridge-ha-service-platform/README.md
```

Final Validation Status

The TLS phase has been validated successfully.

```
cert-manager installed                ✅
cert-manager controllers healthy      ✅
Certificate CRDs installed            ✅
SelfSigned ClusterIssuer READY        ✅
HavenBridge Root CA READY             ✅
Root CA Secret created                ✅
HavenBridge CA ClusterIssuer READY    ✅
havenbridge.lab Certificate READY     ✅
havenbridge-tls Secret created        ✅
Traefik HTTPS listener programmed     ✅
TLS Secret reference resolved         ✅
HTTPS HTTPRoute attached              ✅
```

```
Syrus trusts HavenBridge Root CA       ✅

HTTP redirects to HTTPS                ✅

HTTPS returns HTTP 200                 ✅

FastAPI readiness endpoint reachable   ✅
```

The validated final request was:

```
http://havenbridge.lab/health/ready
        ↓
301 redirect
        ↓
https://havenbridge.lab/health/ready
        ↓
HTTP/2 200
        ↓
{"status":"ready"}
```